INT1339

Ngôn ngữ Lập trình C++
OO – Lớp và Đối tượng

Giảng viên: Lê Hà Thanh

lehathanh@ptithcm.edu.vn

Giới thiệu

- C++ Định nghĩa Lớp
- C++ Định nghĩa Đối tượng
- C++ Các hàm thành viên của lớp
- C++ Bộ thay đổi phương thức truy cập lớp
- C++ Bộ tạo (constructor) và bộ hủy (destructor)
- C++ Hàm bạn bè (friend functions)
- C++ Hàm nội tuyến (inline functions)
- C++ Con trỏ *this*
- Con trỏ đến các lớp trong C++
- Thành viên tĩnh (static members) của lớp trong C++

C++ Kiểu dữ liệu Lớp

- Kiểu dữ liệu do người dùng khai báo
- Biểu diễn một khái niệm (concept) trong mã chương trình,
  - Có các thuộc tính riêng
  - Cách cài đặt
  - Phương tiện ngôn ngữ
- Phân loại:
  - Lớp cụ thể (Concrete classes)
  - Lớp trừu tượng (Abstract classes)
  - Các lớp trong cấu trúc phân tầng

C++ Lớp là gì? (1/2)

- Tập hợp các thành viên, chủ yếu bao gồm:
  - Dữ liệu (data members)
  - Hàm thành viên (member functions)
- Hàm thành viên định nghĩa thao tác khởi tạo đối tượng, sao chép, chuyển và dọn dẹp khi xóa đối tượng thuộc về kiểu lớp
- Toán tử truy cập: **.** (dot) cho đối tượng, **->** cho con trỏ đến đối tượng
- Có thể định nghĩa (lại) các toán tử (vd: **+**, **!**, **[]**) dùng cho lớp

C++ Lớp là gì? (2/2)

- Một lớp là một không gian tên (namespace) chứa các thành viên của lớp
- Các thành viên **public** quy định giao diện của lớp
- Các thành viên **private** quy định chi tiết cài đặt của lớp
- **struct** là kiểu lớp với các thành viên được khai báo mặc định là **public**

C++ Lớp là gì? Ví dụ

- Định nghĩa lớp

```
class X {
private:                    // the representation (implementation) is private
        int m;
public:                     // the user interface is public
        X(int i =0) :m{i} { }   // a constructor (initialize the data member m)

        int mf(int i)       // a member function
        {
                int old = m;
                m = i;      // set a new value
                return old; // return the old value
        }
};
```

C++ Lớp là gì? Ví dụ

- Khai báo đối tượng (biến) thuộc lớp và sử dụng

```
X var {7}; // a variable of type X, initialized to 7

int user(X var, X* ptr)
{
        int x = var.mf(7);          // access using . (dot)
        int y = ptr->mf(9);         // access using -> (arrow)
        int z = var.m;              // error: cannot access private member
}
```

C++ Lớp – Hàm Xử lý Tách biệt

- Định nghĩa khái niệm “ngày” bằng **struct** và các hàm xử lý tách biệt

```
struct Date {          // representation
        int d, m, y;
};

void init_date(Date& d, int, int, int);      // initialize d
void add_year(Date& d, int n);               // add n years to d
void add_month(Date& d, int n);              // add n months to d
void add_day(Date& d, int n);                // add n days to d
```

C++ Lớp – Hàm Thành viên

- struct **Date** với các hàm thành viên

```
struct Date {
        int d, m, y;

        void init(int dd, int mm, int yy);      // initialize
        void add_year(int n);                   // add n years
        void add_month(int n);                  // add n months
        void add_day(int n);                    // add n days
};
```

C++ Lớp – Cách sử dụng

- Khai báo và sử dụng biến đối tượng thuộc lớp

```
Date my_birthday;

void f()
{
        Date today;

        today.init(16,10,1996);
        my_birthday.init(30,12,1950);

        Date tomorrow = today;
        tomorrow.add_day(1);
        // ...
}
```

C++ Lớp – Hàm Thành viên – Cài đặt

- Hàm thành viên của các lớp khác nhau có thể trùng tên → phải chỉ định rõ tên cấu trúc lớp để phân biệt

  <class_name>::<member_function>

- Cài đặt hàm thành viên

```
void Date::init(int dd, int mm, int yy)
{
        d = dd;
        m = mm;
        y = yy;
}
```

C++ Lớp – Hàm Thành viên – Cài đặt

- Bên trong hàm thành viên, tên thành viên có thể được sử dụng trực tiếp, ngầm hiểu là thuộc về đối tượng đang gọi đến hàm thành viên mà không cần tham chiếu tường minh qua tên đối tượng của lớp

- Với **today**,
  m=mm i.e. **today.m=mm**
- Với my_birthday,
  m=mm i.e., **my_birthday.m=mm**

```
void Date::init(int dd, int mm, int yy)
{
        d = dd;
        m = mm;
        y = yy;
}

Date my_birthday;

void f()
{
        Date today;

        today.init(16,10,1996);
        my_birthday.init(30,12,1950);
```

C++ Lớp – Sao chép (Copy)

- **Theo mặc định**, các đối tượng đều có thể được sao chép
- Một đối tượng thuộc lớp được khởi tạo bằng một bản sao từ một đối tượng khác cùng lớp

```
Date d1 = my_birthday;    // initialization by copy
Date d2 {my_birthday};    // initialization by copy
```

- Bản sao của đối tượng chính là bản sao của mọi thành viên thuộc đối tượng
- Có thể được sao chép qua phép gán

```
void f(Date& d)
{
        d = my_birthday;
}
```

C++ Lớp – Kiểm soát Truy xuất Thành viên

- Thành viên **public** được phép truy xuất qua lời gọi từ bên ngoài
- Thành viên **private** chỉ được truy xuất bởi các hàm thành viên của lớp

```
class Date {
        int d, m, y;
public:
        void init(int dd, int mm, int yy);      // initialize

        void add_year(int n);                   // add n years
        void add_month(int n);                  // add n months
        void add_day(int n);                    // add n days
};
```

C++ Lớp – Bảo vệ Thành viên private

```
class Date {
    int d, m, y;
public:
    void init(int dd, int mm, int yy);    // initialize

    void add_year(int n);                 // add n years
    void add_month(int n);                // add n months
    void add_day(int n);                  // add n days
};
```

- Cài đặt hàm thành viên, cho phép truy xuất thành viên **private**:

```
void Date::add_year(int n)
{
    y += n;
}
```

- Lỗi khi truy xuất thành viên **private** từ bên ngoài

```
void timewarp(Date& d)
{
    d.y -= 200;          // error: Date::y is private
}
```

C++ Lớp – Khởi tạo Thành viên bằng Hàm Khởi tạo public

- Dùng hàm thành viên **public** thực hiện các tác vụ khởi tạo cho các thành viên **private**

```
class Date {
    int d, m, y;
public:
    void init(int dd, int mm, int yy);    // initialize

    void add_year(int n);                 // add n years
    void add_month(int n);                // add n months
    void add_day(int n);                  // add n days
};
```

```
Date dx;
dx.m = 3;              // error: m is private
dx.init(25,3,2011);    // OK
```

C++ Lớp – Khai báo Lớp

- Nhắc lại: **struct** là một lớp với toàn bộ thành viên được mặc định là public

```
struct S { /* ... */ };   ≈   class S { public: /* ... */ };
```

- Theo mặc định, các thành viên trong lớp là **private**,
  - Có thể khai báo tường minh bằng bộ chỉ định **private**
- Thành viên public phải được khai báo tường minh là **public**

```
class Date1 {
        int d, m, y;            // private by default
public:
        Date1(int dd, int mm, int yy);
        void add_year(int n);       // add n years
};
```

```
struct Date2 {
private:
        int d, m, y;
public:
        Date2(int dd, int mm, int yy);
        void add_year(int n);       // add n years
};
```

C++ Lớp – Khai báo Lớp

- Không quy định thứ tự khai báo public, private
- Sử dụng thống nhất tạo phong cách lập trình

```
class Date3 {
public:
        Date3(int dd, int mm, int yy);
        void add_year(int n);        // add n years
private:
        int d, m, y;
};
```

```
class Date4 {
public:
        Date4(int dd, int mm, int yy);
private:
        int d, m, y;
public:
        void add_year(int n);        // add n years
};
```

C++ Contructors – Bộ Khởi tạo Đối tượng

- Là hàm thành viên chịu trách nhiệm khởi tạo đối tượng
- Khai báo cùng tên với tên lớp

```
class Date {
        int d, m, y;
public:
        Date(int dd, int mm, int yy);          // constructor
        // ...
};
```

C++ Contructors – Bộ Khởi tạo Đối tượng

- Khi lớp có constructor, các đối tượng của lớp đều được khởi tạo qua lời gọi đến contructor

```
Date today = Date(23,6,1983);
Date xmas(25,12,1990);          // abbreviated form
Date my_birthday;               // error: initializer missing
Date release1_0(10,12);         // error: third argument missing
```

- Dùng ký hiệu **{ }** gọi contructor

```
Date today = Date {23,6,1983};
Date xmas {25,12,1990};         // abbreviated form
Date release1_0 {10,12};        // error: third argument missing
```

C++ Contructors – Nhiều Phương thức Khởi tạo

- Có nhiều cách khởi tạo một đối tượng
- Định nghĩa các constructor khác nhau, tương ứng với từng cách khởi tạo

```
class Date {
        int d, m, y;
public:
        // ...

        Date(int, int, int);        // day, month, year
        Date(int, int);             // day, month, today's year
        Date(int);                  // day, today's month and year
        Date();                     // default Date: today
        Date(const char*);          // date in string representation
};
```

C++ Contructors – Nhiều Phương thức Khởi tạo

- Các constructor phân biệt với nhau qua kiểu dữ liệu của tham số truyền vào contructor
- Bộ dịch sẽ chọn gọi constructor có khai báo tương đồng với lời gọi

```
Date today {4};                    // 4, today.m, today.y
Date july4 {"July 4, 1983"};
Date guy {5,11};                   // 5, November, today.y
Date now;                          // default initialized as today
Date start {};                     // default initialized as today
```

C++ Contructors – Khởi tạo với giá trị mặc định

- Các constructor nhận tham số với giá trị mặc định

```
class Date {
        int d, m, y;
public:
        Date(int dd =0, int mm =0, int yy =0);
        // ...
};

Date::Date(int dd, int mm, int yy)
{
        d = dd ? dd : today.d;
        m = mm ? mm : today.m;
        y = yy ? yy : today.y;

        // check that the Date is valid
}
```

C++ explicit Constructors

- constructor được gọi với lời gọi một đối số hàm ý tới việc chuyển đổi kiểu của đối số sang kiểu định nghĩa bởi lớp

```
complex<double> d {1};          // d=={1,0} (§5.6.2)
```

- Có thể bị trường hợp không xác định rõ ràng kiểu để khởi tạo giá trị cho đối tượng được tạo ra. Ví dụ với lớp **Date**:

```
void my_fct(Date d);

void f()
{
        Date d {15};     // plausible: x becomes {15,today.m,today.y}
        // ...
        my_fct(15);      // obscure
        d = 15;          // obscure
        // ...
}
```

C++ explicit Constructors

- constructor được khai báo **explicit** dùng để khởi tạo đối tượng và thực hiện chuyển kiểu tường minh

```
class Date {
        int d, m, y;
public:
        explicit Date(int dd =0, int mm =0, int yy =0);
        // ...
};

Date d1 {15};               // OK: considered explicit
Date d2 = Date{15};         // OK: explicit
Date d3 = {15};             // error: = initialization does not do implicit conversions
Date d4 = 15;               // error: = initialization does not do implicit conversions

void f()
{
        my_fct(15);         // error: argument passing does not do implicit conversions
        my_fct({15});       // error: argument passing does not do implicit conversions
        my_fct(Date{15});   // OK: explicit
        // ...
}
```

C++ explicit Constructors

- Chú ý: chỉ cần khai báo **explicit** một lần

```
class Date {
        int d, m, y;
public:
        explicit Date(int dd);
        // ...
};

Date::Date(int dd) { /* ... */ }                    // OK
explicit Date::Date(int dd) { /* ... */ }           // error
```

C++ explicit Constructors

- Khai báo **explicit** cho constructor có danh sách đối số tùy ý

```
struct X {
        explicit X();
        explicit X(int,int);
};

X x1 = {};              // error: implicit
X x2 = {1,2};           // error: implicit

X x3 {};                // OK: explicit
X x4 {1,2};             // OK: explicit

int f(X);

int i1 = f({});         // error: implicit
int i2 = f({1,2});      // error: implicit

int i3 = f(X{});        // OK: explicit
int i4 = f(X{1,2});     // OK: explicit
```

C++ Định nghĩa Hàm trong Lớp

- Hàm thành viên có thể được định nghĩa ngay trong lớp – gọi là hàm inline

```
class Date {
public:
        void add_month(int n) { m+=n; }      // increment the Date's m
        // ...
private:
        int d, m, y;
};
```

- Thân hàm ngắn gọn, dễ đọc-hiểu

C++ Định nghĩa Hàm trong Lớp

- Nên tách việc cài đặt hàm thành viên với khai báo hàm thành viên
- Phong cách cài đặt

```
class Date {
public:
        void add_month(int n) { m+=n; }      // increment the Date's m
        // ...
private:
        int d, m, y;
};

inline void Date::add_month(int n)  // add n months
{
        m+=n;      // increment the Date's m
}
```

C++ - Tính Biến đổi của Đối tượng

- Đối tượng có thể được định nghĩa là một “hằng” đối tượng – đối tượng **const**
- Định nghĩa các hàm thành viên cho các đối tượng **const**

C++ - Hàm thành viên “hằng”

- Hàm thành viên được chỉ định **const** không làm thay đổi trạng thái của đối tượng thuộc lớp
- Ví dụ: Khai báo

```
class Date {
        int d, m, y;
public:
        int day() const { return d; }
        int month() const { return m; }
        int year() const;

        void add_year(int n);        // add n years
        // ...
};
```

C++ - Hàm thành viên “hằng”

- Ví dụ: Lỗi thay đổi trạng thái đối tượng

```
int Date::year() const
{
        return ++y;      // error: attempt to change member value in const function
}
```

- Chú ý: phải có tiền tố **const** khi định nghĩa hàm thành viên bên ngoài khai báo lớp

C++ - Hàm thành viên "hằng" – Quy tắc gọi

- Hàm thành viên **const** có thể được gọi bởi các đối tượng **const** và không-**const**
- Hàm thành viên không-**const** chỉ có thể được gọi bởi đối tượng không-**const**.

```
void f(Date& d, const Date& cd)
{
        int i = d.year();        // OK
        d.add_year(1);           // OK

        int j = cd.year();       // OK
        cd.add_year(1);          // error: cannot change value of a const Date
}
```

C++ - Sự Biến đổi trong "hằng"

- Hàm thành viên, về logic, là **const** nhưng có nhu cầu cần thay đổi giá trị của biến thành viên
- Ví dụ:

```
class Date {
public:
        // ...
        string string_rep() const;        // string representation
private:
        bool cache_valid;
        string cache;
        void compute_cache_value(); // fill cache
        // ...
};
```

C++ - mutable

- Định nghĩa: một thành viên là **mutable**, nghĩa là nó có thể bị thay đổi ngay cả khi đối tượng là **const**.
- Ví dụ: khai báo sau

```
class Date {
public:
        // ...
        string string_rep() const;                  // string representation
private:
        mutable bool cache_valid;
        mutable string cache;
        void compute_cache_value() const;           // fill (mutable) cache
        // ...
};
```

C++ - mutable - Cài đặt

- Định nghĩa **string_rep()** :

```
string Date::string_rep() const
{
        if (!cache_valid) {
                compute_cache_value();
                cache_valid = true;
        }
        return cache;
}
```

C++ - mutable – Sử dụng

- Dùng **string_rep()** cho đối tượng **const** và không-**const** như sau:

```
void f(Date d, const Date cd)
{
        string s1 = d.string_rep();
        string s2 = cd.string_rep();            // OK!
        // ...
}
```

C++ Tự tham chiếu

- Trả về một tham chiếu đến đối tượng
- Xâu chuỗi lời gọi
- Ví dụ: để thêm giá trị ngày, tháng, năm vào đối tượng

```
void f(Date& d)
{
        // ...
        d.add_day(1).add_month(1).add_year(1);
        // ...
}
```

C++ Tự tham chiếu

- Khai báo trả về một tham chiếu cho hàm thành viên

```
class Date {
        // ...

        Date& add_year(int n);     // add n years
        Date& add_month(int n); // add n months
        Date& add_day(int n);      // add n days
};
```

C++ Tự tham chiếu – con trỏ *this

- Hàm thành viên non-**static** được gọi cho đối tượng nào (this) thì sẽ tham chiếu đến đối tượng đó (trỏ tới bởi **this**)

```
Date& Date::add_year(int n)
{
        if (d==29 && m==2 && !leapyear(y+n)) {  // beware of February 29
                d = 1;
                m = 3;
        }
        y += n;
        return *this;
}
```

C++ Tự tham chiếu – con trỏ *this

- Trong hàm thành viên non-**const** của một class **X**, kiểu của **this** là **X***
- Trong hàm thành viên **const** của **X**, kiểu của **this** là **const X***
- **this** thường được hiểu ngầm (implicit) khi tham chiếu đến các thành viên non-**static** của một lớp
- Ví dụ: tham chiếu thực tế được hiểu như sau

```
Date& Date::add_year(int n)
{
        if (this->d==29 && this->m==2 && !leapyear(this->y+n)) {
                this->d = 1;
                this->m = 3;
        }
        this->y += n;
        return *this;
}
```

C++ Truy xuất đến Thành viên của Lớp

- Dùng **.** (dot) cho đối tượng
- Dùng **->** cho con trỏ đến đối tượng

```
struct X {
        void f();
        int m;
};

void user(X x, X* px)
{
        m = 1;          // error: there is no m in scope
        x.m = 1;        // OK
        x->m = 1;       // error: x is not a pointer
        px->m = 1;      // OK
        px.m = 1;       // error: px is a pointer
}
```

C++ Truy xuất đến Thành viên của Lớp

- Gọi trực tiếp tên thành viên nếu đang ở trong phạm vi khai báo lớp.
- Nhắc lại: **this** được hiểu ngầm trỏ tới đối tượng đang tương tác

```
void X::f()
{
        m = 1;          // OK: "this->m = 1;"
}
```

- Hàm thành viên có thể gọi tên thành viên khác trước khi thành viên này được khai báo

```
struct X {
        int f() { return m; }   // fine: return this X's m
        int m;
};
```

C++ Truy xuất đến Thành viên của Lớp

- Nếu truy xuất đến một thành viên của lớp mà không cần quan tâm đến đối tượng cụ thể nào, dùng

  <class_name>::<member_name>

```
struct S {
        int m;
        int f();
        static int sm;
};

int X::f() { return m; }            // X's f
int X::sm {7};                      // X's static member sm
int (S::*) pmf() {&S::f};           // X's member f
```

C++ Thành viên "tĩnh" – static Members

- Biến thành viên **static**: Biến thành viên thuộc phạm vi Lớp, không thuộc phạm vi của bất kỳ đối tượng của lớp
  - Chỉ có một bản sao duy nhất của biến **static**
- Hàm thành viên **static**: Hàm truy xuất đến các thành viên của lớp nhưng không cần gọi thông qua đối tượng được tạo ra từ lớp
- Ví dụ:

```
class Date {
        int d, m, y;
        static Date default_date;
public:
        Date(int dd =0, int mm =0, int yy =0);
        // ...
        static void set_default(int dd, int mm, int yy); // set default_date to Date(dd,mm,yy)
};
```

C++ Thành viên "tĩnh" – static Members

- Sử dụng biến thành viên **static**

```
Date::Date(int dd, int mm, int yy)
{
        d = dd ? dd : default_date.d;
        m = mm ? mm : default_date.m;
        y = yy ? yy : default_date.y;

        // ... check that the Date is valid ...
}
```

- Sử dụng hàm thành viên **static**

```
void f()
{
        Date::set_default(4,5,1945);    // call Date's static member set_default()
}
```

C++ Thành viên "tĩnh" – static Members

- Định nghĩa thành viên **static**
- Chú ý: không dùng từ khóa **static** khi định nghĩa thành viên **static**

```
Date Date::default_date {16,12,1770};          // definition of Date::default_date

void Date::set_default(int d, int m, int y)    // definition of Date::set_default
{
        default_date = {d,m,y};                // assign new value to default_date
}
```

C++ Thành viên “tĩnh” – static Members

- Dùng tên lớp để trực tiếp lấy giá trị của biến thành viên static, không cần viết riêng hàm truy xuất.

```
Date copy_of_default_date = Date{};

void f(Date);

void g()
{
        f(Date{});
}
```

C++ Thành viên "tĩnh" – static Members

- Nếu kiểu mục tiêu là rõ ràng thì dùng {} thay cho tên kiểu
- Ví dụ:

```
void f1(Date);

void f2(Date);
void f2(int);

void g()
{
        f1({});          // OK: equivalent to f1(Date{})
        f2({}):          // error: ambiguous: f2(int) or f2(Date)?
        f2(Date{});      // OK
```

C++ Các Kiểu Thành Viên

- Tên kiểu và tên bí danh của kiểu có thể là thành viên của lớp.

```
template<typename T>
class Tree {
        using value_type = T;                          // member alias
        enum Policy { rb, splay, treeps };             // member enum
        class Node {                                   // member class
                Node* right;
                Node* left;
                value_type value;
        public:
                void f(Tree*);
        };
        Node* top;
public:
        void g(const T&);
        // ...
};
```

C++ Các Kiểu Thành Viên

- Một lớp thành viên – lớp lồng nhau (nested class) có thể tham chiếu đến các kiểu và thành viên static của lớp chứa nó.
- Chỉ có thể tham chiếu đến các thành viên non-static khi được cung cấp một đối tượng của lớp bao quanh để tham chiếu đến.

C++ Các Kiểu Thành Viên

- Một lớp lồng nhau có thể truy cập đến các thành viên của lớp bao, bao gồm cả các thành viên private nhưng không có khái niệm về đối tượng hiện tại của lớp bao ngoài này.
- Ví dụ:

```
template<typename T>
void Tree::Node::f(Tree* p)
{
        top = right;                    // error: no object of type Tree specified
        p->top = right;                 // OK
        value_type v = left->value;     // OK: value_type is not associated with an object
}
```

C++ Kiểu Lớp Cụ thể (Concrete classes)

- Tương đương kiểu dữ liệu sẵn có trong C++,
- Có cú pháp và tập các toán tử riêng
  - Ví dụ: kiểu số phức, kiểu số nguyên có độ chính xác bất kỳ được dùng như kiểu int
- Có cách trình bày dữ liệu đặc thù
  - Ví dụ vector hay string được xem như là mở rộng từ kiểu mảng (array)

C++ Các Kiểu Thành Viên

- Chú ý: một lớp không có quyền truy xuất đến các thành viên của lớp lồng nhau bên trong nó.

```
template<typename T>
void Tree::g(Tree::Node* p)
{
        value_type val = right->value;          // error: no object of type Tree::Node
        value_type v = p->right->value;         // error: Node::right is private
        p->f(this);                             // OK
}
```

C++ Concret Classes – Một ví dụ

```
namespace Chrono {

    enum class Month { jan=1, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec };

    class Date {
    public:             // public interface:
        class Bad_date { };  // exception class

        explicit Date(int dd ={}, Month mm ={}, int yy ={});          // {} means "pick a default"

    // nonmodifying functions for examining the Date:
        int day() const;
        Month month() const;
        int year() const;

        string string_rep() const;                  // string representation
        void char_rep(char s[], in max) const;      // C-style string representation
```

C++ Concret Classes – Một ví dụ

```
// (modifying) functions for changing the Date:
        Date& add_year(int n);                          // add n years
        Date& add_month(int n);                         // add n months
        Date& add_day(int n);                           // add n days
private:
        bool is_valid();                                // check if this Date represents a date
        int d, m, y;                                    // representation
};

bool is_date(int d, Month m, int y);                    // true for valid date
bool is_leapyear(int y);                                // true if y is a leap year

bool operator==(const Date& a, const Date& b);
bool operator!=(const Date& a, const Date& b);

const Date& default_date();                             // the default date

ostream& operator<<(ostream& os, const Date& d);        // print d to os
istream& operator>>(istream& is, Date& d);              // read Date from is into d
} // Chrono
```

Ví dụ: Cách sử dụng lớp Date

```
void f(Date& d)
{
        Date lvb_day {16,Month::dec,d.year()};

        if (d.day()==29 && d.month()==Month::feb) {
                // ...
        }

        if (midnight()) d.add_day(1);

        cout << "day after:" << d+1 << '\n';

        Date dd;   // initialized to the default date
        cin>>dd;
        if (dd==d) cout << "Hurray!\n";
}
```

C++ Concret classes – Hàm Thành viên

```
Date::Date(int dd, Month mm, int yy)
        :d{dd}, m{mm}, y{yy}
{
        if (y == 0) y = default_date().year();
        if (m == Month{}) m = default_date().month();
        if (d == 0) d = default_date().day();

        if (!is_valid()) throw Bad_date();
}
```

C++ - Các hàm trợ giúp

- Cách gọi khác: các hàm tiện ích
- C: Gom nhóm trong một tập tin và được gọi khi cần
  #include “<interface_file>.h”
- C++: khai báo một namespace
- Ví dụ:

```
namespace Chrono {          // facilities for dealing with time

    class Date { /* ... */};

    int diff(Date a, Date b);
    bool is_leapyear(int y);
    bool is_date(int d, Month m, int y);
    const Date& default_date();
    Date next_weekday(Date d);
    Date next_saturday(Date d);
    // ...
}
```

C++ - Các hàm trợ giúp

- Định nghĩa hàm trợ giúp

```
bool Chrono::is_date(int d, Month m, int y)
{
	int ndays;

	switch (m) {
	case Month::feb:
		ndays = 28+is_leapyear(y);
		break;
	case Month::apr: case Month::jun: case Month::sep: case Month::nov:
		ndays = 30;
		break;
	case Month::jan: case Month::mar: case Month::may: case Month::jul:
	case Month::aug: case Month::oct: case Month::dec:
		ndays = 31;
		break;
	default:
		return false;
	}

	return 1<=d && d<=ndays;
}
```

C++ - Các hàm trợ giúp

- Sử dụng

```
const Date& Chrono::default_date()
{
        static Date d {1,Month::jan,1970};
        return d;
}
```

C++ Các Toán tử chồng lấn (Overloaded Op.)

- Định nghĩa các toán tử để áp dụng cho một kiểu dữ liệu cụ thể
- Ví dụ, toán tử **operator==()** để so sánh bằng (**==**) cho kiểu **Date**

```
inline bool operator==(Date a, Date b)          // equality
{
        return a.day()==b.day() && a.month()==b.month() && a.year()==b.year();
}
```

C++ Các Toán tử chồng lấn (Overloaded Op.)

- Định nghĩa các toán tử khác

```
bool operator!=(Date, Date);          // inequality
bool operator<(Date, Date);           // less than
bool operator>(Date, Date);           // greater than
// ...

Date& operator++(Date& d) { return d.add_day(1); }          // increase Date by one day
Date& operator--(Date& d) { return d.add_day(-1); }         // decrease Date by one day

Date& operator+=(Date& d, int n) { return d.add_day(n); }   // add n days
Date& operator-=(Date& d, int n) { return d.add_day(-n); }  // subtract n days

Date operator+(Date d, int n) { return d+=n; }              // add n days
Date operator-(Date d, int n) { return d+=n; }              // subtract n days

ostream& operator<<(ostream&, Date d);                      // output d
istream& operator>>(istream&, Date& d);                     // read into d
```

Hết phần này